

Community

 Web API

- Overview
- Getting started
- Concepts >
- ▼ Concepts
- < Concepts
- Access Token
- API calls
- Apps
- Authorization
- Redirect URIs
- Playlists
- Quota modes
- Rate limits
- Scopes
- Spotify URIs and IDs
- Track Relinking

- Tutorials >
- ▶ Tutorials
- How-Tos >
- ▶ How-Tos
- REFERENCE
- Albums >

API calls

The Spotify Web API is a restful API with different endpoints which return JSON metadata about music artists, albums, and tracks, directly from the Spotify Data Catalogue.

Base URL

The base address of Web API is `https://api.spotify.com`.

Authorization

All requests to Spotify Web API require authorization. Make sure you have read the [authorization](#) guide to understand the basics.

To access private data through the Web API, such as user profiles and playlists, an application must get the user's permission to access the data.

Requests

Data resources are accessed via standard HTTP requests in UTF-8 format to an API endpoint. The Web API uses the following HTTP verbs:

Method	Action
GET	Retrieves resources
POST	Creates resources
PUT	Changes and/or replaces resources or collections

Method	Action
DELETE	Deletes resources

Responses

Web API normally returns JSON in the response body. Some endpoints (e.g [Change Playlist Details](#)) don't return JSON but the HTTP status code

Response Status Codes

Web API uses the following response status codes, as defined in the [RFC 2616](#) and [RFC 6585](#):

Status Code	Description
200	OK - The request has succeeded. The client can read the result of the request in the body and the headers of the response.
201	Created - The request has been fulfilled and resulted in a new resource being created.
202	Accepted - The request has been accepted for processing, but the processing has not been completed.
204	No Content - The request has succeeded but returns no message body.
304	Not Modified. See Conditional requests .
400	Bad Request - The request could not be understood by the server due to malformed syntax. The message body will contain more information; see Response Schema .
401	Unauthorized - The request requires user authentication or, if the request included

Status Code	Description
	authorization credentials, authorization has been refused for those credentials.
403	Forbidden - The server understood the request, but is refusing to fulfill it.
404	Not Found - The requested resource could not be found. This error can be due to a temporary or permanent condition.
429	Too Many Requests - Rate limiting has been applied.
500	Internal Server Error. You should never receive this error because our clever coders catch them all ... but if you are unlucky enough to get one, please report it to us through a comment at the bottom of this page.
502	Bad Gateway - The server was acting as a gateway or proxy and received an invalid response from the upstream server.
503	Service Unavailable - The server is currently unable to handle the request due to a temporary condition which will be alleviated after some delay. You can choose to resend the request again.

Response Error

Web API uses two different formats to describe an error:

- Authentication Error Object
- Regular Error Object

Authentication Error Object

Whenever the application makes requests related to authentication or authorization to Web API, such as retrieving an access token or refreshing an access token, the error response follows [RFC 6749](#) on the OAuth 2.0 Authorization Framework.

Key	Value Type	Value Description
error	string	A high level description of the error as specified in RFC 6749 Section 5.2 .
error_description	string	A more detailed description of the error as specified in RFC 6749 Section 4.1.2.1 .

Here is an example of a failing request to refresh an access token.

```
1 $ curl -H "Authorization: Basic Yjc...cK" -d
2
3 {
4   "error": "invalid_client",
5   "error_description": "Invalid client sec
6 }
```

Regular Error Object

Apart from the response code, unsuccessful responses return a JSON object containing the following information:

Key	Value Type	Value Description
status	integer	The HTTP status code that is also returned in the response header. For further information, see Response Status Codes .

Key	Value Type	Value Description
message	string	A short description of the cause of the error.

Here, for example is the error that occurs when trying to fetch information for a non-existent track:

```
1 $ curl -i "https://api.spotify.com/v1/tracks"
2
3 HTTP/1.1 400 Bad Request
4 {
5   "error": {
6     "status": 400,
7     "message": "invalid id"
8   }
9 }
```

Conditional Requests

Most API responses contain appropriate cache-control headers set to assist in client-side caching:

- If you have cached a response, do not request it again until the response has expired.
- If the response contains an ETag, set the If-None-Match request header to the ETag value.
- If the response has not changed, the Spotify service responds quickly with **304 Not Modified** status, meaning that your cached version is still good and your application should use it.

Timestamps

Timestamps are returned in [ISO 8601](#) format as [Coordinated Universal Time \(UTC\)](#) with a zero offset: YYYY-MM-DDTHH:MM:SSZ. If the time is imprecise (for example, the date/time of an album release), an

additional field indicates the precision; see for example, `release_date` in an [Album Object](#).

Pagination

Some endpoints support a way of paging the dataset, taking an offset and limit as query parameters:

```
1 $ curl
2 https://api.spotify.com/v1/artists/1vCWHaC5f
```

In this example, in a list of 50 (total) singles by the specified artist : From the twentieth (offset) single, retrieve the next 10 (limit) singles.

DOCUMENTATION

[Web API](#)[Web Playback SDK](#)[Ads API](#)[iOS](#)[Android](#)[Embeds](#)[Commercial Hardware](#)

GUIDELINES

[Design](#)[Accessibility](#)

COMMUNITY

[News](#)[Forum](#)

LEGAL

[Developer Terms](#)[Developer Policy](#)[Compliance Tips](#)

Third Party Licenses

[Legal](#) [Cookies](#) © 2025 Spotify AB